Polynomial System Solving via Factor Branching and Recursive Splitting

Ilay Menahem[a], Michael Shalyt[a] and Ido Kaminer[a,b]

[a]Andrew and Erna Viterbi Faculty of Electrical and Computer Engineering, Technion – Israel Institute of Technology, Haifa, 3200003, Israel
[b]Department of Materials Science and Engineering, Technion – Israel Institute of Technology, Haifa, 3200003, Israel

ARTICLE INFO

Keywords:
Polynomial systems
Gröbner bases
Factorization
Minimal hitting sets
Recursive splitting

ABSTRACT

Polynomial systems are a fundamental language for describing algebraic and geometric problems in mathematics and its applications. Beyond finding individual solutions, one often seeks global properties of the solution space, such as its dimension and irreducible components, or structural properties and decompositions of the underlying polynomial ideal. For large polynomial systems with many variables and high degrees, determining these global properties can become intractable. Leading approaches include variants of Buchberger's algorithm with syzygy, resultants, and cylindrical algebraic decomposition. However, even the most modern approaches still often fail on practical problems or slight perturbations of standard benchmarks. Here we introduce a framework that divides a polynomial system into simpler subproblems by interreducing and factoring its equations. The framework only assumes that the solver used on each subproblem describes satisfiable (SAT) branches and certifies unsatisfiable (UNSAT) ones. When individual branch computations exceed time limits, the framework refines stalled branches through recursive variable splitting, illustrated here by zeroing ($x = 0$ versus $x \neq 0$) to isolate simpler sub-varieties. In our experiments, we use Gröbner-basis solvers and apply the method to a system arising from research in quantum field theory for which no standard method in our comparison succeeds. Together, these ideas offer a path toward the exact analysis of previously inaccessible polynomial systems.

1. Introduction

Systems of multivariate polynomial equations appear whenever a symbolic model contains nonlinear algebraic constraints. Applications include robotics, coding theory, optimization, statistics, and physics (Sturmfels, 2002). In many of these problems, an isolated solution is not enough. One needs the dimension of the solution set, its irreducible components, or the exact relations that hold on every solution. These are questions about the variety and its defining ideal, rather than about a single point (Cox et al., 2015; Kreuzer and Robbiano, 2000).

Elimination by resultants provided an early systematic way to attack polynomial systems, but the introduction of Gröbner bases gave a common algorithmic framework for elimination, ideal membership, dimension, and related tasks (Cox et al., 2015; Kreuzer and Robbiano, 2000). Buchberger developed this framework in his 1965 thesis (Buchberger, 2006). Later algorithms changed how the expensive reductions are organized. F4 replaces individual polynomial reductions with sparse linear algebra, while F5 uses signatures to avoid many reductions to zero (Faugère, 1999, 2002). Primary-decomposition algorithms use Gröbner bases and factorization to separate components (Gianni et al., 1988). Factorizing Gröbner-basis methods, split on individual polynomial factorizations and preserve the solution set; earlier methods also interreduce and prune redundant branches (Decker et al., 2025; Gräbe, 1995). Numerical algebraic geometry takes a different route: homotopy continuation computes isolated roots numerically, while witness sets represent positive-dimensional components (Sommese et al., 2001; Sommese and Wampler, 2005).

Recent work has also explored machine-learning methods for polynomial systems. Peifer et al. trained a reinforcement-learning policy for pair selection in Buchberger's algorithm (Peifer et al., 2020). Kera et al. trained transformers to compute Gröbner bases directly for a restricted class of zero-dimensional radical ideals (Kera et al., 2024).

Despite this progress, large systems can remain inaccessible.

Here, we introduce a solver-agnostic framework that breaks down a polynomial system into smaller subproblems by interreducing and factoring its equations. The framework enumerates the minimal hitting sets of the resulting factor sets; each hitting set contains at least one factor from every equation and therefore defines one branch of the original solution set. An exact branch solver then either describes that branch, including any positive-dimensional families,

or certifies that it is UNSAT. Since the worst-case complexity of computing a Gröbner basis is doubly exponential in the number of variables (Mayr and Meyer, 1982), breaking down a large system into smaller subproblems can be substantially faster than solving the original system directly.

If a factor branch remains too expensive for the chosen solver, the framework can refine it by splitting a selected variable into the cases $x = 0$ and $x \neq 0$. This heuristic split can expose additional algebraic structure for the branch solver without losing any solutions. While this is one of many possible splitting criteria, it proves particularly effective on the quantum field theory problem that we investigate.

We prove that refinement and variable zeroing terminate and preserve the full solution set. We compare the resulting implementation with direct Gröbner-basis computation, symbolic decomposition, and numerical homotopy on generated benchmarks. We also apply the framework to a system from quantum field theory: with the `facstd` backend, the framework completes the computation, whereas standard standalone solvers timed out.

2. Preliminaries and notation

2.1. Polynomial systems, ideals, and varieties

Let K be a field, let $\overline{K}$ be its algebraic closure, and write $R = K[x_1, \dots, x_n]$. The input is a finite system $P = \{p_1, \dots, p_m\} \subseteq R$ of nonzero polynomials. Unless stated otherwise, a solution is a point in $\overline{K}^n$. For a finite set $Q \subseteq R$, its solution set is

$$V(Q) = \{a \in \overline{K}^n : q(a) = 0 \text{ for every } q \in Q\}.$$

We call Q satisfiable (SAT) if $V(Q) \neq \emptyset$ and unsatisfiable (UNSAT) if $V(Q) = \emptyset$.

The ideal generated by Q, denoted $\langle Q \rangle$, consists of all sums $\sum_{q \in Q} h_q q$ with $h_q \in R$.

For $X \subseteq \overline{K}^n$, its vanishing ideal $I(X)$ contains every polynomial in R that vanishes on all of X. The radical of an ideal J is $\sqrt{J} = \{g \in R : g^r \in J \text{ for some } r \geq 1\}$. Hilbert's Nullstellensatz gives

$$I(V(Q)) = \sqrt{\langle Q \rangle}.$$

Thus $V(Q) = \emptyset$ exactly when $\langle Q \rangle = \langle 1 \rangle$. These facts connect solution sets with computations in polynomial ideals (Cox et al., 2015; Kreuzer and Robbiano, 2000).

2.2. Monomial orders and Gröbner bases

Let $\mathbb{M}_n = \{x^\alpha : \alpha \in \mathbb{N}^n\}$ be the monomials of R. A monomial order $\prec$ is a total order on $\mathbb{M}_n$ such that every nonempty subset has a least element and

$$u \prec v \implies uw \prec vw \qquad (u, v, w \in \mathbb{M}_n).$$

For a nonzero polynomial p, the leading monomial $\mathrm{LM}(p)$ is the greatest monomial of p under $\prec$, $\mathrm{LC}(p)$ is its coefficient, and $\mathrm{LT}(p)$ is its leading term.

For an ideal J, define its leading-monomial ideal by

$$\mathrm{LM}(J) = \langle \mathrm{LM}(f) : f \in J \setminus \{0\} \rangle.$$

A finite set $G \subseteq J$ is a Gröbner basis of J if

$$\mathrm{LM}(J) = \langle \mathrm{LM}(g) : g \in G \rangle.$$

A finite set Q is *interreduced* if every element is monic and no term of one element is divisible by the leading monomial of another. Algorithm 1 interreduces a finite set of nonzero polynomials.

Algorithm 1 Interreduction

Require: A finite set $Q \subseteq R \setminus \{0\}$ and a monomial order $\prec$
Ensure: An interreduced set that generates $\langle Q \rangle$

while there exist distinct $p, q \in Q$ and a term cu of p with $\mathrm{LM}(q) \mid u$ **do**
  $r \leftarrow p - \dfrac{cu}{\mathrm{LT}(q)} q$
  $Q \leftarrow Q \setminus \{p\}$
  if $r \neq 0$ **then**
    $Q \leftarrow Q \cup \{r\}$
  end if
end while
$Q \leftarrow \{p/\mathrm{LC}(p) : p \in Q\}$
return Q

Each loop iteration cancels one monomial u and introduces only monomials strictly smaller than u. The multiset extension of the monomial order is well-founded, so the loop terminates. Interreduction preserves $\langle Q \rangle$ and $V(Q)$.

Every ideal has a unique reduced Gröbner basis for a fixed monomial order. The reduced basis is $\{1\}$ if and only if $J = \langle 1 \rangle$. Therefore, a Gröbner basis computation can describe a branch and detect when it is UNSAT (Cox et al., 2015; Kreuzer and Robbiano, 2000).

An UNSAT core of a system Q is a subset $U \subseteq Q$ with $V(U) = \emptyset$. An UNSAT core is minimal if there is no $U' \subsetneq U$ such that $V(U') = \emptyset$. Any system containing U is also UNSAT, since adding equations cannot introduce solutions.

2.3. Factor sets and hitting sets

Fix a factorization of each input polynomial over K:

$$p_i = a_i \prod_{f \in C_i} f^{e_{i,f}},$$

where $a_i \in K \setminus \{0\}$, C_i contains the distinct monic, nonconstant irreducible factors of p_i, and $e_{i,f} \geq 1$. Multiplicity does not affect the zero set. A point satisfies $p_i = 0$ exactly when at least one factor in C_i vanishes at that point.

A hitting set for $C_1, \ldots, C_m$ is a set $H \subseteq \bigcup_i C_i$ such that $H \cap C_i \neq \emptyset$ for every i. It is minimal if no proper subset is also a hitting set. We write $\mathcal{H}(P)$ for the family of minimal hitting sets obtained from P. Each $H \in \mathcal{H}(P)$ defines a branch in which all factors in H vanish (Berge, 1989).

2.4. Encoding nonzero variables

Let $N \subseteq \{x_1, \ldots, x_n\}$ be the variables required to be nonzero. Here are two ways to encode these constraints as polynomial equations.

One encoding introduces a separate auxiliary variable s_x for each $x \in N$ and works in the extended ring $R_{\mathrm{sep}} = R[s_x : x \in N]$. For each $x \in N$, define the polynomial

$$\sigma_x = 1 - s_x x, \tag{1}$$

and impose the equation $\sigma_x = 0$.

The second encoding uses one auxiliary variable s, works in $R_{\mathrm{prod}} = R[s]$, and combines all nonzero constraints into the polynomial

$$\sigma_N = 1 - s \prod_{x \in N} x, \tag{2}$$

with the equation $\sigma_N = 0$.

Let $Q \subseteq R$ and let π denote the appropriate coordinate projection onto the original $x_1, \ldots, x_n$ coordinates. In the first encoding,

$$\pi\left(V\left(Q \cup \{\sigma_x : x \in N\}\right)\right) = \{a \in V(Q) : a_x \neq 0 \text{ for every } x \in N\}.$$

Indeed, $\sigma_x = 0$ has a solution s_x exactly when $a_x \neq 0$. Similarly, the product encoding satisfies

$$\pi\left(V\left(Q \cup \{\sigma_N\}\right)\right) = \{a \in V(Q) : a_x \neq 0 \text{ for every } x \in N\}.$$

3. Factor branching framework

3.1. Framework description

Polynomial-ideal computations can be intractable even for small systems (Mayr and Meyer, 1982). Thus, decomposing a system into smaller and simpler subproblems may make the individual solver calls easier.

We break a system into smaller subproblems by alternating two operations: interreducing the current subproblem, and factoring each generator to enumerate the minimal hitting sets of the factor sets. Then again do the same on each hitting set, until the subproblems can't be further broken down.

Here is the pseudocode for this refinement of a polynomial system Q into a smaller set of subproblems.

Algorithm 2 Refinement

Require: A finite polynomial system Q and a collection A of normalized systems visited on the current recursion path

$Q \leftarrow Q \setminus \{0\}$
$\widetilde{Q} \leftarrow \textsc{Interreduce}(Q)$ ▷ Optional; otherwise $\widetilde{Q} \leftarrow Q$
Factor each generator of $\widetilde{Q}$
$\mathcal{H} \leftarrow$ the minimal hitting sets of the factor sets
$T \leftarrow \emptyset$
for all $H \in \mathcal{H}$ **do**
    if $H = \widetilde{Q}$ or $H \in A$ **then**
        $T \leftarrow T \cup \{H\}$
    else
        $T \leftarrow T \cup \textsc{Refine}(H, A \cup \{\widetilde{Q}\})$
    end if
end for
return T

Let $\mathcal{L}(P)$ denote the terminal hitting sets obtained from P. For each $H \in \mathcal{L}(P)$, the branch solver must either describe $V(H)$ exactly or certify that $V(H) = \emptyset$.

Algorithm 3 Factor branching

Require: A set of polynomials $P = \{p_1, \ldots, p_m\}$ and an exact branch solver $\mathcal{B}$.

$S \leftarrow \emptyset$ ▷ Solution-set representations
$R \leftarrow \textsc{Refine}(P, \emptyset)$
for all $H \in R$ **do**
    Use $\mathcal{B}$ to solve H
    if $\mathcal{B}$ returns a representation of $V(H)$ **then**
        Add the returned representation to S
    end if
end for
return S

The order of the terminal hitting sets affects running time but not correctness. Optionally, we can sort the hitting sets by their estimated cost to solve. We use the number of generators in a hitting set as a proxy for the cost, processing smaller hitting sets first.

3.2. UNSAT-core learning

We also evaluated UNSAT-core learning, following ideas from polynomial UNSAT-core extraction and conflict-driven learning (Marques-Silva et al., 2009; Marques-Silva and Sakallah, 1999; Sun et al., 2016). After certifying a branch H as UNSAT, this extension extracts a subset $U \subseteq H$ with $V(U) = \emptyset$ by iterative refinement as in Section 5 of (Sun et al., 2016). Any later branch containing U can be discarded, since every solution of that branch would also have to solve U. However, UNSAT-core learning provided no performance benefit on the systems we tested. We therefore omit it from the algorithms presented here.

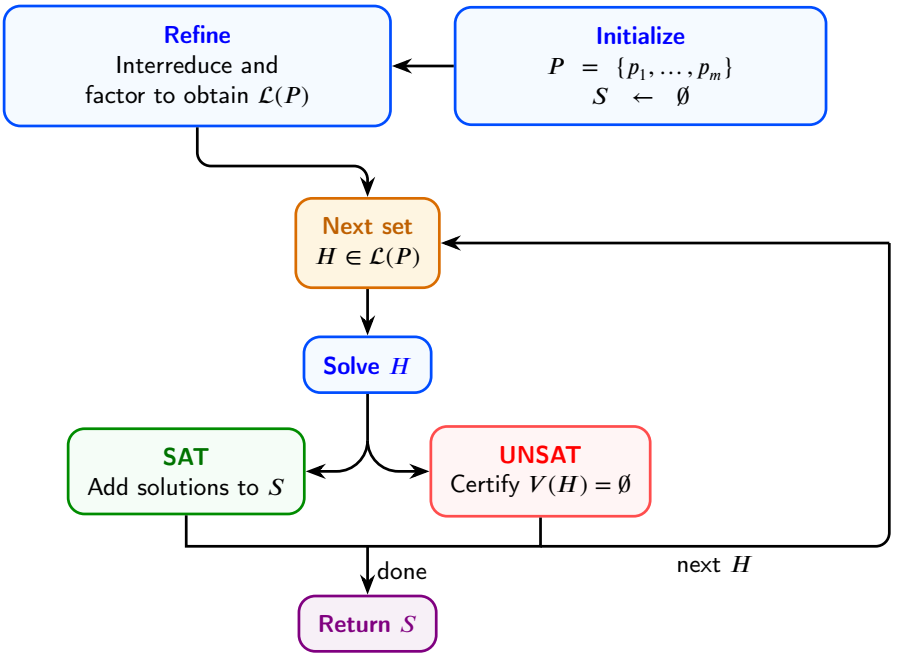


Figure 1: Factor branching procedure of Algorithm 3. Refinement produces terminal hitting-set branches, which the exact solver either represents or certifies as UNSAT.

3.3. Theoretical analysis

Let S be a collection of solution-set representations, and $\mathrm{Sol}(S)$ be the union of the represented sets.

For a system Q, write $\mathcal{H}(Q)$ for the minimal hitting sets obtained in one factorization round.

Theorem 1 (One-round branch decomposition).

$$V(Q) = \bigcup_{H \in \mathcal{H}(Q)} V(H).$$

PROOF. Write $Q = \{q_1, \ldots, q_m\}$ and let F_i be the nonconstant irreducible factors of q_i. Take $x \in V(Q)$. Since $q_i(x) = 0$, at least one factor in F_i vanishes at x for every i. The set of all factors that vanish at x is a hitting set of $F_1, \ldots, F_m$. Because the factor sets are finite, this set contains a minimal hitting set H. Every factor in H vanishes at x, so $x \in V(H)$.

For the reverse inclusion, take $x \in V(H)$ for some $H \in \mathcal{H}(Q)$. The set H intersects every F_i, so each q_i has a factor that vanishes at x. Hence $q_i(x) = 0$ for every i, and $x \in V(Q)$. □

Lemma 1 (Recursive refinement). *Suppose refinement of Q terminates with the finite collection $\mathcal{L}(Q)$. Then*

$$V(Q) = \bigcup_{H \in \mathcal{L}(Q)} V(H).$$

PROOF. Consider the finite recursion tree produced by Algorithm 2. For every internal node G, let $\widetilde{G}$ be its interreduction (or G itself when interreduction is disabled). Interreduction preserves the generated ideal, so $V(G) = V(\widetilde{G})$. Theorem 1 then gives

$$V(G) = \bigcup_{H \in \mathcal{H}(\widetilde{G})} V(H),$$

where the sets in $\mathcal{H}(\widetilde{G})$ are precisely the children of G. A bottom-up induction on the finite tree shows that the variety of each node equals the union of the varieties of the terminal nodes in its subtree. Applying this statement at the root Q proves the result. □

Theorem 2 (Termination of refinement). *Algorithm 2 terminates on every finite polynomial system.*

PROOF. If interreduction is disabled, every child consists of irreducible polynomials. Factoring such a child again returns it unchanged, so the condition $H = \widetilde{Q}$ makes it terminal.

Suppose interreduction is enabled. For an interreduced system Q, let

$$M(Q) = \langle \mathrm{LM}(q) : q \in Q \rangle.$$

For every hitting-set child H and every $q \in Q$, some $h \in H$ divides q. Hence $\mathrm{LM}(h)$ divides $\mathrm{LM}(q)$ and $M(Q) \subseteq M(H)$.

Suppose equality holds. For any $h \in H$, choose $q \in Q$ having h as a factor. Equality gives some $q' \in Q$ such that $\mathrm{LM}(q') \mid \mathrm{LM}(h) \mid \mathrm{LM}(q)$. Because Q is interreduced, this is possible only for $q' = q$. The two outer leading monomials are therefore equal, so the remaining factor of q is a unit. Thus h and q are associates, and their monic normalization gives $h = q$. Applying this argument to the factor selected for every $q \in Q$ gives $H = Q$, which is terminal.

It remains to account for the next interreduction. In an elementary reduction, the old leading monomial either remains, or it is already divisible by the leading monomial of the reducer. Consequently, interreduction cannot shrink the ideal generated by the current leading monomials. Thus, for every nonterminal child H and its interreduction $\widetilde{H}$,

$$M(Q) \subsetneq M(H) \subseteq M(\widetilde{H}).$$

Therefore M strictly increases along every recursive path.

The ascending-chain condition for monomial ideals rules out an infinite path. Every node has finitely many children, so König's lemma implies that the refinement tree is finite. Therefore Algorithm 2 terminates. □

The following theorem allows the use of Algorithm 3 without loss of correctness.

Theorem 3. *Algorithm 3 is sound and complete for P.*

PROOF. By Theorem 2, refinement terminates with a finite collection $\mathcal{L}(P)$. Every interreduction round preserves the current ideal, and every factorization round preserves the zero set by Theorem 1. Lemma 1 therefore gives

$$V(P) = \bigcup_{H \in \mathcal{L}(P)} V(H).$$

Consider any $H \in \mathcal{L}(P)$. The exact solver either returns an exact representation of $V(H)$, which the algorithm adds to S, or certifies that $V(H) = \emptyset$. Empty branches therefore make no contribution, while every represented branch contributes exactly its variety. Hence

$$\mathrm{Sol}(S) = \bigcup_{H \in \mathcal{L}(P)} V(H) = V(P).$$

Every representation added to S is exact, which proves soundness, and the equality proves completeness. □

Finally, we show that the factor branching algorithm terminates.

Theorem 4. *If the exact solver is known to terminate, then Algorithm 3 terminates.*

PROOF. By Theorem 2, the call $R = \textsc{Refine}(P, \emptyset)$ terminates and returns a finite collection. Therefore Algorithm 3 calls the exact solver a finite number of times, and each call terminates by assumption. Thus Algorithm 3 terminates. □

4. Recursive splitting

4.1. Factor branching with variable zeroing

Suppose the solver does not terminate within a reasonable amount of time on a hitting set produced by Algorithm 3.

Then we can further simplify the system by recursive splitting. In each child branch, we assume a side condition and run Algorithm 3 on the simplified system. This process continues until the solver terminates or the computation budget is exhausted.

For completeness, the side conditions must cover all possibilities. They should also simplify the system for the solver input and work well with Algorithm 2.

In this work, we use *variable zeroing*: recursive splitting with the side conditions $x = 0$ and $x \neq 0$ for a selected variable x. Our experience with the physical systems considered here suggests that a variable is often always zero or always nonzero.

Algorithm 4 Factor branching with variable zeroing

Require: A polynomial system P and a timeout τ
function ZEROINGBRANCHING(P, τ)
  $X \leftarrow$ VARIABLES(P)
  return SOLVEBYZEROING($P, X, \emptyset, \emptyset, \tau$)
end function
function SOLVEBYZEROING(Q, X, Z, N, τ)
  $(S, T) \leftarrow$ FACTORBRANCHINGWITHTIMEOUT(Q, τ)
  $U \leftarrow \emptyset$ ▷ Unresolved branches
  for all $R \in T$ **do**
    if $X = \emptyset$ **then**
      $U \leftarrow U \cup \{(R, Z, N)\}$
    else
      $x \leftarrow$ CHOOSEVARIABLE(R, X)
      $R_0 \leftarrow$ SUBSTITUTE($R, x, 0$)
      $R_1 \leftarrow$ UPDATESATURATION($R, N \cup \{x\}$)
      $(S_0, U_0) \leftarrow$ SOLVEBYZEROING($R_0, X \setminus \{x\}, Z \cup \{x\}, N, \tau$)
      $(S_1, U_1) \leftarrow$ SOLVEBYZEROING($R_1, X \setminus \{x\}, Z, N \cup \{x\}, \tau$)
      $S \leftarrow S \cup S_0 \cup S_1$
      $U \leftarrow U \cup U_0 \cup U_1$
    end if
  end for
  return (S, U)
end function

Let N be the variables declared nonzero on the current path. Subsection 2.4 describes two ways to encode these conditions; the experiments use Equation (1). Since refinement may rewrite each σ_x, UpdateSaturation(R, N') removes every generator of R containing an auxiliary variable and adds σ_x for $x \in N'$.

Lemma 2. *Refinement keeps exactly one generator containing s_x, of the form $c(1 - s_x h_x)$ with $h_x \equiv x$ modulo the auxiliary-free generators R'. Hence $\pi(V(R)) = \{a \in V(R') : a_x \neq 0,\ x \in N\}$, and* UpdateSaturation *loses no constraint.*

PROOF. The leading monomial of $1 - s_x h_x$ contains s_x, so it reduces no other generator, is reduced only by R' (changing h_x by an element of $\langle R' \rangle$), and, being linear in s_x, does not factor. □

Solutions of the nonzero branch are projected onto the original variables, so the auxiliary variables do not appear in the output. The zero and nonzero decisions remain part of the solution-set representation.

For comparison, we repeat the experiment twice: once with the fixed side conditions $x = 1$ and $x \neq 1$, and once with each split independently drawing an integer a uniformly from $\{-10, \ldots, 10\}$ and using the side conditions $x = a$ and $x \neq a$. Both runs have a ten-minute wall-clock limit. Neither alternative finishes within ten minutes, and in both cases the search has branches that remain unresolved even after exhausting every variable.

These comparisons demonstrate that zero/nonzero splits are substantially more effective than arbitrary affine splittings on these structured systems.

4.2. Choice of variable for splitting

The choice of variable for splitting affects the efficiency of the algorithm. If a variable barely occurs in the current hitting set, recursive calls may repeat the same work. A variable that occurs in many generators is a more promising choice. Setting it to zero removes all terms containing it at once. In the nonzero branch, UpdateSaturation adds the equation $\sigma_x = 0$.

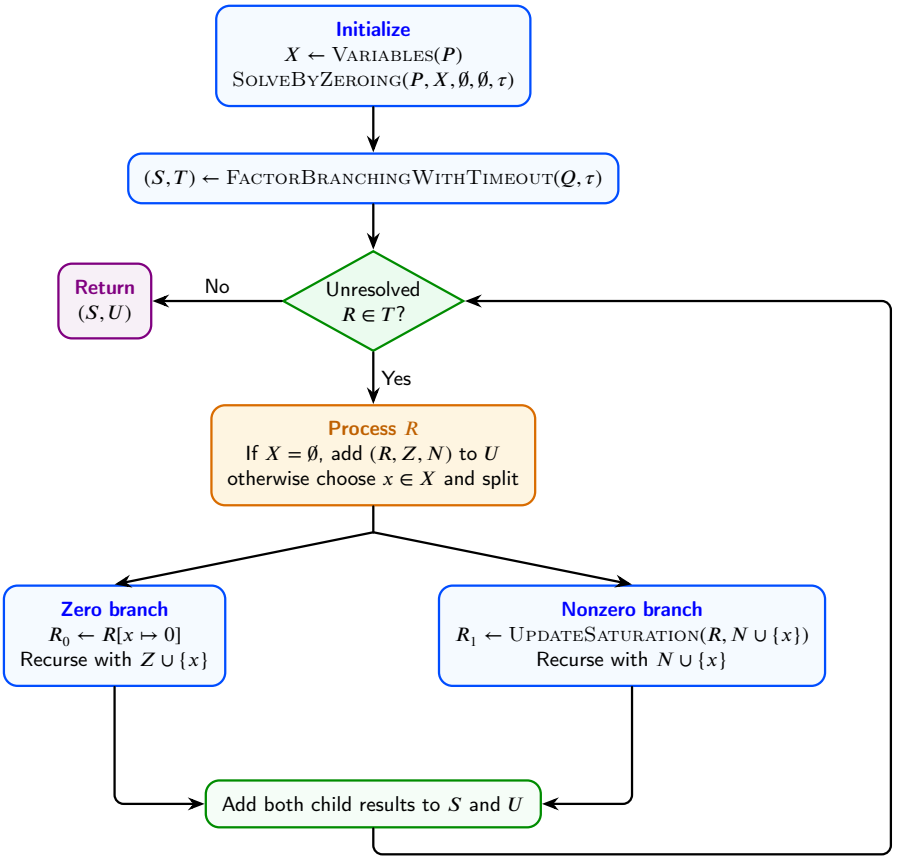


Figure 2: Flowchart of the recursive variable-zeroing procedure in Algorithm 4.

For each candidate variable x that is still available on the current branch, define

$$g_Q(x) = \#\{q \in Q : x \text{ occurs in } q\},$$
$$t_Q(x) = \#\{(q, m) : q \in Q,\ m \text{ is a term of } q,\ \deg_x(m) > 0\},$$
$$e_Q(x) = \sum_{q \in Q} \sum_{\substack{m \text{ a term of } q, \\ \deg_x(m) > 0}} \deg_x(m).$$

Its score is the triple

$$h_Q(x) = \big(g_Q(x), t_Q(x), e_Q(x)\big),$$

and scores are compared lexicographically.

4.3. Branch contraction

Let $\mathcal{H}_0$ and $\mathcal{H}_1$ be the minimal hitting sets for $x = 0$ and $x \neq 0$. Suppose, up to exchanging the two children, that

$$\mathcal{H}_0 = \emptyset \qquad \text{and} \qquad \mathcal{H}_1 = \{H\}.$$

The first child is UNSAT. Theorem 1 and the variable-zeroing split give

$$V_Y(Q; Z, N) = V_Y(H; Z, N \cup \{x\}).$$

There is no need to run the branch solver on H before choosing the next splitting variable.

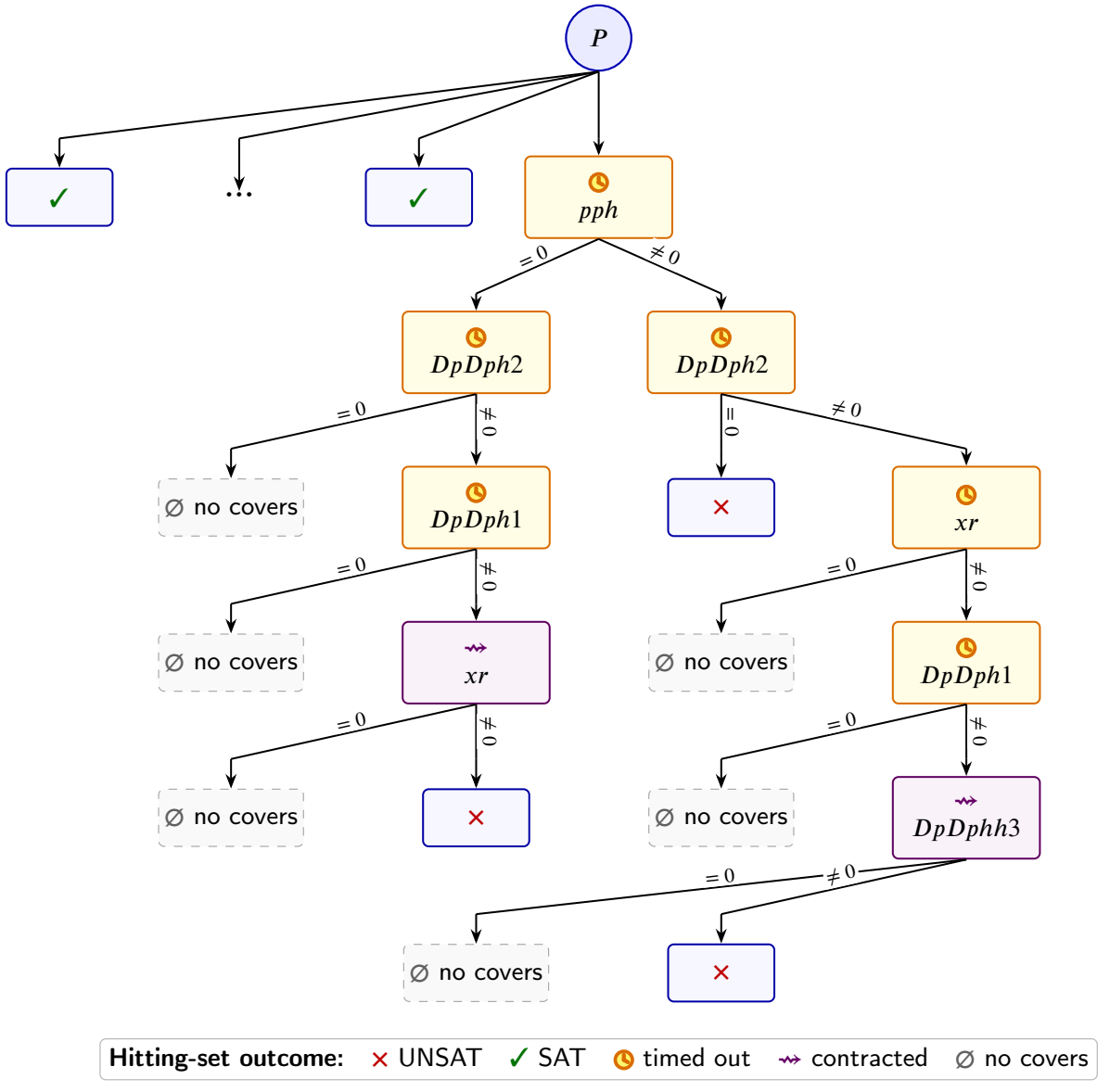


Figure 3: Recursion tree of factor branching with variable zeroing on the fifth-order graviton QFT system (detailed in Section 5.3) using M4GB with a five-second timeout per hitting set. The run takes 33.768 seconds.

4.4. Theoretical analysis

The correctness argument uses only that the side conditions cover all possibilities. Let Y be the original variables, let A be the conditions on the current branch, and let $V_Y(Q; A)$ be the projection onto Y of the solutions of Q satisfying A. For variable zeroing, we write $V_Y(Q; Z, N)$ for the conditions $z = 0$ for $z \in Z$ and $y \neq 0$ for $y \in N$.

Theorem 5 (Correctness of a covering split). *Let $D_1, \ldots, D_k$ be side conditions such that every point satisfying A satisfies at least one D_i. If Q_i encodes Q together with D_i, then*

$$V_Y(Q; A) = \bigcup_{i=1}^{k} V_Y(Q_i; A \cup \{D_i\}).$$

PROOF. Every solution on the left satisfies some D_i and therefore belongs to the corresponding set on the right. Every set on the right imposes all equations and conditions from the parent branch, so the reverse inclusion also holds. □

For variable zeroing, the conditions $x = 0$ and $x \neq 0$ cover all values of x. With $Q_0 = Q|_{x=0}$ and $Q_1 = \text{UpdateSaturation}(Q, N \cup \{x\})$, the theorem gives

$$V_Y(Q; Z, N) = V_Y(Q_0; Z \cup \{x\}, N) \cup V_Y(Q_1; Z, N \cup \{x\}). \tag{3}$$

If factor branching with a timeout returns solved representations S and timed-out hitting sets T, the proof of Theorem 3 gives

$$V_Y(Q; A) = \text{Sol}(S) \cup \bigcup_{R \in T} V_Y(R; A). \tag{4}$$

Theorem 6 (Correctness of recursive splitting). *Suppose every split uses side conditions that cover all possibilities. If the recursive procedure returns (S, U) on input P, where U contains the unresolved pairs (R, A_R), then*

$$V(P) = \text{Sol}(S) \cup \bigcup_{(R, A_R) \in U} V_Y(R; A_R).$$

PROOF. Equation (4) accounts for the solved and timed-out hitting sets at each call. An unresolved hitting set is either added to U or replaced by child branches. Theorem 5 shows that any covering family of child conditions preserves its solution set. Applying this argument recursively proves the result. □

Corollary 1 (Completeness of variable zeroing). *If $U = \emptyset$ in the output of Algorithm 4, then* $\text{Sol}(S) = V(P)$.

Theorem 7 (Termination of variable zeroing). *Algorithm 4 terminates if each timed solver call returns after τ.*

PROOF. Algorithm 2 terminates by Theorem 2 and returns finitely many hitting sets. Each variable-zeroing split removes one variable from the finite set X, so every recursive path is finite. Therefore the whole recursion tree is finite. □

5. Experiments

5.1. Setup

We implemented Algorithm 3 in Rust 1.97.1 and used Symbolica 1.5 for polynomial arithmetic over the rationals (Ruijl, 2023). In all experiments, we solve the branches with a Gröbner-basis computation. The available implementations are M4GB, F5, GVW, F4, and Singular 4.4.1 (Decker et al., 2025; Faugère, 1999, 2002; Gao et al., 2016; Makarim and Stevens, 2017). A basis containing 1 certifies that the branch is UNSAT. The comparison also includes six direct whole-ideal Gröbner-basis configurations, three decomposition methods from Singular, Macaulay2 1.26.06, and a numerical method from HomotopyContinuation.jl under Julia 1.12.6 (Bezanson et al., 2017; Breiding and Timme, 2018; Decker et al., 2025; Grayson and Stillman, 2026).

For Algorithm 4, we test M4GB, F5, GVW, F4, and Singular as branch solvers, disable UNSAT-core learning, and impose a three-second limit on each hitting-set computation. We selected this cutoff by trying one, two, three, and five seconds on the hard gated systems. Three seconds is the smallest tested value that lets Singular complete the Katsura-8 residual (about 2.6 seconds); five seconds completes no additional hard system and only delays the subsequent splits.

We ran the benchmark on an Apple M4 Pro with 14 CPU cores and 24 GB of memory. Every table entry is a single run. Every method received 60 seconds per system; the three-second variable zeroing limit is an inner, per-hitting-set limit within that budget.

For factor branching and variable zeroing, T is end-to-end wall time and F is the time to first SAT branch, measured from the start of the minimal-hitting-set search. Factorization and input processing are therefore included in T but not in F. The complete measurements are stored in the appendix.

5.2. Baseline systems

The baseline has 20 generated systems in six families.

- Signed-distance cycles and affine-factor hypergraphs test many inexpensive hitting sets.
- The overlapping random multiples use seeded dense quadratic factors that recur across equations, with no factor shared by every equation.
- For gated systems, we create a new variable and multiply every equation in Cyclic, Katsura, or dense quadratic by it (Björck and Fröberg, 1991; Böge et al., 1986).
- This produces one easy selector hitting set and one nonlinear residual hitting set.

Family	Tested sizes	Variables/equations; hitting sets
Signed-distance cycle	$n = 8, 10, 12$	$8/8, 10/10, 12/12$; $8, 8, 32$
Overlapping random multiples	$(n, p, k) = (4, 6, 2), (6, 8, 3), (8, 10, 3)$	n/n; $4, 16, 28$
Positive affine hypergraph	$(n, m, k) = (8, 6, 4), (10, 8, 5), (12, 10, 6)$	n/m; $63, 239, 833$
Gated Cyclic	orders $5, 6, 7$	$6/5, 7/6, 8/7$; 2 each
Gated Katsura and dense quadratics	Katsura $6, 7, 8$; dense $n = 5, 6, 7$	$8/7, 9/8, 10/9, 6/5, 7/6, 8/7$; 2 each
Square affine hypergraph	$(n, k) = (8, 4), (10, 5)$	n/n; $31, 304$

Table 1
The generated systems used for the baseline benchmark.

The overlapping-random-multiple runs disable interreduction so that the generated factor-incidence pattern reaches the hitting-set enumerator unchanged. All other factor branching runs use interreduction. Table 2 selects the fastest completed factor branching backend and the fastest completed method in each baseline class. A factor branching entry gives T/F in seconds; the other entries give total wall time. "Timeout" means that every tested method in that class timed out.

System	Best factor branching T/F	Best direct GB	Best decomposition	Numerical
Signed distance, $n = 8$	F4 0.008/0.000	F4 0.008	facstd 0.190	32.410
Signed distance, $n = 10$	F4 0.025/0.002	F4 0.029	facstd 0.430	29.944
Signed distance, $n = 12$	F4 0.087/0.002	F4 0.472	primdec 3.056	36.880
Overlapping $(4, 6, 2)$	F5 0.008/0.000	F4 0.413	facstd 0.265	30.069
Overlapping $(6, 8, 3)$	F5 0.132/0.001	timeout	timeout	timeout
Overlapping $(8, 10, 3)$	Singular 2.931/0.010	timeout	timeout	timeout
Positive affine $(8, 6, 4)$	F5 0.008/0.000	timeout	facstd 0.368	50.934
Positive affine $(10, 8, 5)$	F5 0.020/0.000	timeout	facstd 1.836	timeout
Positive affine $(12, 10, 6)$	F5 0.067/0.000	timeout	facstd 53.318	timeout
Gated Cyclic-5	F4 0.009/0.000	F4 0.008	facstd 0.181	35.054
Gated Cyclic-6	Singular 0.045/0.008	Singular 0.309	M2 2.620	37.314
Gated Cyclic-7	timeout	timeout	timeout	58.287
Gated Katsura-6	Singular 0.049/0.008	GVW 0.226	facstd 0.233	31.532
Gated Katsura-7	Singular 0.283/0.008	GVW 3.176	facstd 0.539	31.980
Gated Katsura-8	Singular 2.657/0.052	timeout	facstd 3.596	34.264
Gated dense, $n = 5$	Singular 0.029/0.008	M4GB 0.171	facstd 0.198	29.289
Gated dense, $n = 6$	Singular 0.546/0.008	M4GB 4.736	facstd 0.755	30.053
Gated dense, $n = 7$	Singular 42.020/0.017	timeout	facstd 48.820	32.261
Square affine $(8, 4)$	Singular 0.470/0.023	timeout	facstd 0.601	timeout
Square affine $(10, 5)$	F4 1.174/0.000	timeout	facstd 2.141	timeout
#systems solved	19	10	17	14
#fastest (ties counted)	17	2	0	2

Table 2
Best completed baseline results across solver classes. All times are in seconds.

Factor branching is a good choice when factorization produces branch ideals that are substantially easier than the expanded input ideal. A large number of hitting sets can still be manageable when individual branches are cheap to solve; the positive affine family shows that the hitting-set count alone is not decisive. The method is less effective when one hitting set retains the difficulty of the original nonlinear system, as Cyclic-7 does.

The measurements of time to first SAT branch support this explanation. On positive affine $(12, 10, 6)$, F rounds to 0.000 seconds despite the 833 hitting sets. On gated Katsura-8, the first SAT branch is found in 0.052 seconds, but the full run takes 2.657 seconds. Factorization therefore finds the easy component quickly, while the nonlinear residual controls completion time.

5.3. Physical System

The system is an order-five approximation constraining graviton couplings in asymptotic-safety models to remain consistent with quantum field theory (QFT) (Rafie-Zinedine, 2018; Vacca and Zanusso, 2010). Solving this system helps determine admissible fixed-point structures in quantum gravity (Vacca and Zanusso, 2010).

For this larger system, every run, including the standalone baselines, has a 120-second total limit. Within it, variable zeroing uses a five-second limit per hitting set; factor branching places no further limit on a hitting set.

Backend	Factor branching T/F (s)	Factor branching result	Variable zeroing T/F (s)	Variable zeroing result
M4GB	*timeout*/0.005	partial: 32 SAT, 1 timeout	34.053/0.452	complete: 8 splits
F5	*timeout*/0.003	partial: 32 SAT, 1 timeout	*timeout*/0.457	partial: 24 splits, 8 unresolved
GVW	*timeout*/0.006	partial: 32 SAT, 1 timeout	*timeout*/0.463	partial: 24 splits, 8 unresolved
F4	*timeout*/0.003	partial: 32 SAT, 1 timeout	*timeout*/0.471	partial: 23 splits, 7 unresolved
Singular	*timeout*/0.010	partial: 32 SAT, 1 timeout	27.750/0.470	complete: 7 splits
`facstd`	1.341/0.010	complete: 32 SAT, 1 UNSAT	1.566/0.470	complete: 0 splits

Table 3
Physical-system results with a 120-second total limit per run; variable zeroing also uses a 5-second limit per hitting set. Both methods refine the system into the same 33 terminal hitting sets; the factor branching result column counts them by outcome. A split is one $x = 0$ versus $x \neq 0$ case split; an unresolved branch is one still open when the total limit expires. The ten standalone baselines, including standalone `facstd`, all time out and are omitted.

All ten standalone baselines time out after 120 seconds on the physical system: M4GB, F5, GVW, and F4 in grevlex order, Singular and Macaulay2 lex Gröbner bases, Singular `facstd` and `primdecGTZ`, Macaulay2 `decompose`, and HomotopyContinuation.jl. Factor branching with `facstd` completes in 1.341 seconds. Every backend solves the same 32 of the 33 terminal hitting sets as SAT within 0.13 seconds in total, and times out on the last one. Only `facstd` shows that this hitting set is UNSAT, so only in hindsight do we know that the 32 SAT branches found by the other backends already contain every solution.

Variable zeroing also completes with M4GB and Singular, making completion more robust to backend choice on this system: three of six backends succeed, versus one with factor branching alone.

6. Conclusion

We introduced a solver-agnostic framework that decomposes polynomial systems into smaller branches by enumerating minimal hitting sets of their factor sets. The framework can recursively split timed-out branches into the cases $x = 0$ and $x \neq 0$.

We proved that refinement terminates and preserves the full solution set. Variable zeroing also preserves the solution set and terminates provided that each timed solver call returns. The output is complete whenever no branches remain unresolved.

On the baseline benchmarks, factor branching completed 19 of 20 systems, compared with 10 for direct Gröbner-basis methods, 17 for decomposition methods, and 14 for the numerical method. On the QFT system, variable zeroing extends certified completion from `facstd` alone to M4GB and Singular, improving robustness to backend choice: three of six backends complete, versus one with factor branching alone.

Future work will test this benefit on other systems, and improve the variable choice for splitting or find better splitting side conditions.

CRediT authorship contribution statement

Ilay Menahem: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Michael Shalyt:** Writing – review & editing. **Ido Kaminer:** Writing – review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code and data associated with this work are publicly available at https://github.com/IlayMenahem/soothing.

Acknowledgments

This work did not receive any dedicated funding, and the authors have no one to acknowledge beyond those credited as authors above.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used generative AI tools to assist with drafting and editing portions of the manuscript text. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Berge, C., 1989. Hypergraphs: Combinatorics of Finite Sets. North-Holland, Amsterdam.

Bezanson, J., Edelman, A., Karpinski, S., Shah, V.B., 2017. Julia: A fresh approach to numerical computing. SIAM Review 59, 65–98. doi:10.1137/141000671.

Björck, G., Fröberg, R., 1991. A faster way to count the solutions of inhomogeneous systems of algebraic equations, with applications to cyclic *n*-roots. Journal of Symbolic Computation 12, 329–336. doi:10.1016/S0747-7171(08)80153-8.

Böge, W., Gebauer, R., Kredel, H., 1986. Some examples for solving systems of algebraic equations by calculating Gröbner bases. Journal of Symbolic Computation 2, 83–98. doi:10.1016/S0747-7171(86)80014-1.

Breiding, P., Timme, S., 2018. HomotopyContinuation.jl: A package for homotopy continuation in Julia, in: Mathematical Software – ICMS 2018, Springer. pp. 458–465. doi:10.1007/978-3-319-96418-8_54.

Buchberger, B., 2006. Bruno Buchberger's PhD thesis 1965: An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal. Journal of Symbolic Computation 41, 475–511. doi:10.1016/j.jsc.2005.09.007.

Cox, D.A., Little, J., O'Shea, D., 2015. Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra. 4 ed., Springer, Cham. doi:10.1007/978-3-319-16721-3.

Decker, W., Greuel, G.M., Pfister, G., Schönemann, H., 2025. Singular 4-4-1: A computer algebra system for polynomial computations. https://www.singular.uni-kl.de.

Faugère, J.C., 1999. A new efficient algorithm for computing Gröbner bases (F4). Journal of Pure and Applied Algebra 139, 61–88. doi:10.1016/S0022-4049(99)00005-5.

Faugère, J.C., 2002. A new efficient algorithm for computing Gröbner bases without reduction to zero (F5), in: Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation, ACM. pp. 75–83. doi:10.1145/780506.780516.

Gao, S., Volny, IV, F., Wang, M., 2016. A new framework for computing Gröbner bases. Mathematics of Computation 85, 449–465. doi:10.1090/MCOM/2969.

Gianni, P., Trager, B., Zacharias, G., 1988. Gröbner bases and primary decomposition of polynomial ideals. Journal of Symbolic Computation 6, 149–167. doi:10.1016/S0747-7171(88)80040-3.

Gräbe, H.G., 1995. On factorized Gröbner bases, in: Fleischer, J., Grabmeier, J., Hehl, F.W., Küchlin, W. (Eds.), Computer Algebra in Science and Engineering. World Scientific, Singapore, pp. 77–89. URL: https://www.informatik.uni-leipzig.de/~graebe/ComputerAlgebra/Publications/fgb.pdf.

Grayson, D.R., Stillman, M.E., 2026. Macaulay2, a software system for research in algebraic geometry. https://macaulay2.com. Version 1.26.06.

Kera, H., Ishihara, Y., Kambe, Y., Vaccon, T., Yokoyama, K., 2024. Learning to compute Gröbner bases, in: Advances in Neural Information Processing Systems. URL: https://openreview.net/forum?id=ZRz7XlxBzQ.

Kreuzer, M., Robbiano, L., 2000. Computational Commutative Algebra 1. Springer, Berlin. doi:10.1007/978-3-540-70628-1.

Makarim, R.H., Stevens, M., 2017. M4GB: An efficient Gröbner-basis algorithm, in: Proceedings of the 2017 ACM International Symposium on Symbolic and Algebraic Computation, ACM. pp. 293–300. doi:10.1145/3087604.3087638.

Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Handbook of Satisfiability. IOS Press. volume 185 of *Frontiers in Artificial Intelligence and Applications*, pp. 131–153. doi:10.3233/978-1-58603-929-5-131.

Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521. doi:10.1109/12.769433.

Mayr, E.W., Meyer, A.R., 1982. The complexity of the word problems for commutative semigroups and polynomial ideals. Advances in Mathematics 46, 305–329. doi:10.1016/0001-8708(82)90048-2.

Peifer, D., Stillman, M., Halpern-Leistner, D., 2020. Learning selection strategies in Buchberger's algorithm, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 7575–7585. URL: https://proceedings.mlr.press/v119/peifer20a.html.

Rafie-Zinedine, S., 2018. Simplifying Quantum Gravity Calculations. Master's thesis. Lund University. ArXiv:1808.06086.

Ruijl, B., 2023. Symbolica: Modern computer algebra. https://symbolica.io. Version 1.5.

Sommese, A.J., Verschelde, J., Wampler, C.W., 2001. Numerical decomposition of the solution sets of polynomial systems into irreducible components. SIAM Journal on Numerical Analysis 38, 2022–2046. doi:10.1137/S0036142900372549.

Sommese, A.J., Wampler, II, C.W., 2005. The Numerical Solution of Systems of Polynomials Arising in Engineering and Science. World Scientific, Singapore. doi:10.1142/5763.

Sturmfels, B., 2002. Solving Systems of Polynomial Equations. volume 97 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI. doi:10.1090/cbms/097.

Sun, X., Ilioaea, I., Kalla, P., Enescu, F., 2016. Finding unsatisfiable cores of a set of polynomials using the Gröbner basis algorithm, in: Principles and Practice of Constraint Programming, Springer. pp. 859–875. doi:10.1007/978-3-319-44953-1_54.

Vacca, G.P., Zanusso, O., 2010. Asymptotic safety in einstein gravity and scalar-fermion matter. Physical Review Letters 105, 231601. doi:10.1103/PhysRevLett.105.231601.

A. Complete experimental results

The tables below give every run summarized in Table 2. Unless stated otherwise, entries are end-to-end wall times in seconds. P marks a partial run and TO an outer timeout. These are single measurements.

A.1. Factor branching

Table 4 reports all five branch-solver configurations. A partial entry found the easy component shown in the main text but left at least one hitting set unresolved.

A.2. Variable zeroing

Table 5 gives the complete output of the fastest completed variable zeroing configuration evaluated across the baseline suite. Every backend receives three seconds per hitting set and a 60-second outer budget, and UNSAT-core learning is disabled. The overlapping-random-multiple rows disable interreduction, as in the factor branching experiment. Here N is the number of recursion nodes, E the number of hitting-set evaluations, O/S the numbers of per-hitting-set timeouts and variable zeroing splits, and C/U the numbers of returned components and unresolved branches. The time entry is T/F.

The dense $n = 7$ run failed while constructing the first recursive child: the expanded Singular program exceeded the operating system's command-line argument limit. This is an implementation error, not an UNSAT result or a timeout. We also tried M4GB, F5, GVW, and F4 on this system. These backends execute in-process; after a per-hitting-set timeout their non-cancellable worker threads continued alongside the recursive children until the process was killed under memory pressure. Singular remains preferable for the hard rows because its external worker can be terminated. The Cyclic-7 run exhausted all useful splitting variables on one route and therefore returned one unresolved branch.

A.3. Direct whole-ideal Gröbner bases

A.4. Decomposition and numerical methods

System	M4GB	F5	GVW	F4	Singular
Signed distance, $n = 8$	0.651	0.053	0.122	0.008	0.083
Signed distance, $n = 10$	0.046	10.927	45.797	0.025	0.084
Signed distance, $n = 12$	0.182	6.256	1.841	0.087	0.271
Overlapping $(4, 6, 2)$	0.017	0.008	0.011	0.015	0.038
Overlapping $(6, 8, 3)$	1.053	0.132	0.280	0.657	0.201
Overlapping $(8, 10, 3)$	TO	10.937	30.687	TO	2.931
Positive affine $(8, 6, 4)$	0.009	0.008	0.010	0.008	0.497
Positive affine $(10, 8, 5)$	0.021	0.020	0.028	0.020	1.870
Positive affine $(12, 10, 6)$	0.072	0.067	0.096	0.068	6.498
Gated Cyclic-5	0.010	0.010	0.011	0.009	0.022
Gated Cyclic-6	P 60.010	5.960	2.275	P 60.013	0.045
Gated Cyclic-7	P 60.875	P 60.504	P 60.955	P 60.024	P 60.103
Gated Katsura-6	0.366	0.483	1.338	2.540	0.049
Gated Katsura-7	4.487	15.220	42.113	P 60.012	0.283
Gated Katsura-8	54.055	P 60.325	P 60.047	P 60.012	2.657
Gated dense, $n = 5$	0.177	0.071	0.200	0.204	0.029
Gated dense, $n = 6$	4.716	3.194	7.475	21.498	0.546
Gated dense, $n = 7$	P 60.008	P 60.050	P 60.041	P 60.015	42.020
Square affine $(8, 4)$	15.117	TO	TO	3.096	0.470
Square affine $(10, 5)$	14.750	TO	47.561	1.174	2.548

Table 4
End-to-end wall times for every factor-branching configuration.

System	Backend	T/F	N	E	O/S	C/U	Result
Signed distance, $n = 8$	F4	0.008/0.001	1	8	0/0	8/0	complete
Signed distance, $n = 10$	F4	0.025/0.002	1	8	0/0	8/0	complete
Signed distance, $n = 12$	F4	0.091/0.003	1	32	0/0	32/0	complete
Overlapping $(4, 6, 2)$	F5	0.009/0.001	1	4	0/0	4/0	complete
Overlapping $(6, 8, 3)$	F5	0.132/0.021	1	16	0/0	16/0	complete
Overlapping $(8, 10, 3)$	Singular	2.892/0.152	1	28	0/0	28/0	complete
Positive affine $(8, 6, 4)$	Singular	0.181/0.012	1	4	0/0	4/0	complete
Positive affine $(10, 8, 5)$	F4	0.060/0.007	1	242	0/0	242/0	complete
Positive affine $(12, 10, 6)$	Singular	9.600/0.078	1	511	0/0	511/0	complete
Gated Cyclic-5	F4	0.011/0.001	1	2	0/0	2/0	complete
Gated Cyclic-6	Singular	0.043/0.009	1	2	0/0	2/0	complete
Gated Cyclic-7	Singular	24.118/0.009	15	17	8/7	1/1	partial
Gated Katsura-6	Singular	0.049/0.009	1	2	0/0	2/0	complete
Gated Katsura-7	Singular	0.284/0.009	1	2	0/0	2/0	complete
Gated Katsura-8	Singular	2.588/0.009	1	2	0/0	2/0	complete
Gated dense, $n = 5$	Singular	0.029/0.008	1	2	0/0	2/0	complete
Gated dense, $n = 6$	Singular	0.535/0.008	1	2	0/0	2/0	complete
Gated dense, $n = 7$	Singular	4.797/–	–	–	–	–	error
Square affine $(8, 4)$	Singular	0.445/0.012	1	31	0/0	31/0	complete
Square affine $(10, 5)$	F4	1.326/0.015	1	304	0/0	304/0	complete

Table 5
Selected variable-zeroing result for every system. Times are end-to-end wall time and time to first SAT branch in seconds.

System	M4GB	F5	GVW	F4	Singular	Macaulay2
Signed distance, $n = 8$	0.646	0.014	0.010	0.008	0.333	0.570
Signed distance, $n = 10$	0.130	0.221	0.112	0.029	0.194	0.518
Signed distance, $n = 12$	3.266	20.044	8.685	0.472	0.512	1.293
Overlapping $(4, 6, 2)$	1.000	24.865	0.472	0.413	TO	TO
Overlapping $(6, 8, 3)$	TO	TO	TO	TO	TO	TO
Overlapping $(8, 10, 3)$	TO	TO	TO	TO	TO	TO
Positive affine $(8, 6, 4)$	TO	TO	TO	TO	TO	TO
Positive affine $(10, 8, 5)$	TO	TO	TO	TO	TO	TO
Positive affine $(12, 10, 6)$	TO	TO	TO	TO	TO	TO
Gated Cyclic-5	0.012	0.279	0.009	0.008	0.183	0.480
Gated Cyclic-6	TO	TO	1.738	TO	0.309	0.917
Gated Cyclic-7	TO	TO	TO	TO	TO	TO
Gated Katsura-6	0.436	44.961	0.226	5.277	14.430	TO
Gated Katsura-7	5.165	TO	3.176	TO	TO	TO
Gated Katsura-8	TO	TO	TO	TO	TO	TO
Gated dense, $n = 5$	0.171	8.884	0.179	0.200	0.529	TO
Gated dense, $n = 6$	4.736	TO	6.755	17.472	TO	TO
Gated dense, $n = 7$	TO	TO	TO	TO	TO	TO
Square affine $(8, 4)$	TO	TO	TO	TO	TO	TO
Square affine $(10, 5)$	TO	TO	TO	TO	TO	TO

Table 6
End-to-end wall times for every direct whole-ideal Gröbner-basis configuration.

System	facstd	primdecGTZ	Macaulay2	Homotopy
Signed distance, $n = 8$	0.190	0.320	1.020	32.410
Signed distance, $n = 10$	0.430	0.667	3.224	29.944
Signed distance, $n = 12$	4.800	3.056	25.223	36.880
Overlapping $(4, 6, 2)$	0.265	TO	TO	30.069
Overlapping $(6, 8, 3)$	TO	TO	TO	TO
Overlapping $(8, 10, 3)$	TO	TO	TO	TO
Positive affine $(8, 6, 4)$	0.368	TO	TO	50.934
Positive affine $(10, 8, 5)$	1.836	TO	TO	TO
Positive affine $(12, 10, 6)$	53.318	TO	TO	TO
Gated Cyclic-5	0.181	0.337	0.709	35.054
Gated Cyclic-6	2.729	TO	2.620	37.314
Gated Cyclic-7	TO	TO	TO	58.287
Gated Katsura-6	0.233	TO	TO	31.532
Gated Katsura-7	0.539	TO	TO	31.980
Gated Katsura-8	3.596	TO	TO	34.264
Gated dense, $n = 5$	0.198	TO	TO	29.289
Gated dense, $n = 6$	0.755	TO	TO	30.053
Gated dense, $n = 7$	48.820	TO	TO	32.261
Square affine $(8, 4)$	0.601	TO	TO	TO
Square affine $(10, 5)$	2.141	TO	TO	TO

Table 7
End-to-end wall times for every decomposition and numerical configuration.